C++ COMPLETE REFERENCE LIBRARY • MODULE 12

12 OPERATOR OVERLOADING — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 operator overloading fundamentals and rules
3. **Theory Chapter:** 02 mandatory complex system and overflow analysis
4. **Theory Chapter:** 03 unary and binary member vs friend
5. **Theory Chapter:** 04 special operators stream and assignment
6. **Visual Memory & Concept Diagrams** (2 Vector Diagrams)
7. **Annotated C++ Source Programs** (5 Programs)
8. **Practice Question Bank:** conceptual questions
9. **Practice Question Bank:** debugging questions
10. **Practice Question Bank:** predict the output
11. **Practice Question Bank:** programming questions
12. **Step-by-Step Solutions:** conceptual solutions
13. **Step-by-Step Solutions:** output solutions
14. **Step-by-Step Solutions:** programming solutions

12 — Operator Overloading (Priority Deep-Dive Module)

This module covers C++ Operator Overloading from classroom fundamentals to standard-level edge cases, featuring the mandatory `complex_system` class (`Show_Num()` , `operator+()` , `c1 + c2` , and `c1.operator+(c2)`), unary prefix/postfix overloading, member vs. `friend` overloads, and signed integer overflow rules (`INT_MIN` negation UB).

Directory Contents

1. Theory (`01_THEORY/`)

- `01_operator_overloading_fundamentals_and_rules.md` (./01_THEORY/01_operator_overloading_fundamentals_and_rules.md) — Meaning, intuition, syntax (`operator@`), operators that can and cannot be overloaded, and arity/precedence preservation rules.
- `02_mandatory_complex_system_and_overflow_analysis.md` (./01_THEORY/02_mandatory_complex_system_and_overflow_analysis.md) — **Mandatory**

`complex_system` **Deep-Dive** (`Show_Num()` , `operator+()` , implicit `c1 + c2` vs. explicit `c1.operator+(c2)`), plus signed integer overflow UB and the `INT_MIN` negation edge case when formatting negative imaginary parts.

- `03_unary_and_binary_member_vs_friend.md` (./01_THEORY/03_unary_and_binary_member_vs_friend.md) — Unary prefix (`++obj`) vs. postfix (`obj++` with dummy `int`), member vs. `friend` binary overloads, and why `10 + c1` requires a non-member overload.
- `04_special_operators_stream_and_assignment.md` (./01_THEORY/04_special_operators_stream_and_assignment.md) — Stream insertion/extraction (`<<` , `>>`), copy assignment (`=`), subscript (`[]`), and C++20/23 three-way comparison (`⟺`).

2. Programs (`02_PROGRAMS/`)

- `01_classroom_complex_system.cpp` (./02_PROGRAMS/01_classroom_complex_system.cpp) — Category A: **Mandatory** `complex_system` **Classroom Program** demonstrating constructor initialization, `Show_Num()` , `operator+()` , `c1 + c2` , and `c1.operator+(c2)` .
- `02_complex_system_overflow_edge_case.cpp` (./02_PROGRAMS/02_complex_system_overflow_edge_case.cpp) — Category D / Safe Alternative: Demonstration and safe fix for signed integer overflow (`INT_MIN` negation) in `complex_system` .
- `03_unary_prefix_and_postfix_overload.cpp` (./02_PROGRAMS/03_unary_prefix_and_postfix_overload.cpp) — Category A: Overloading unary `-` , prefix `++` , and postfix `++(int)` in a classroom `counter` class.
- `04_friend_binary_and_stream_overload.cpp` (./02_PROGRAMS/04_friend_binary_and_stream_overload.cpp) — Category A: Overloading `operator+` and `operator<<` using `friend` functions.
- `05_invalid_operator_overload_compile_error.cpp` (./02_PROGRAMS/05_invalid_operator_overload_compile_error.cpp) — Category B: Intentional compile-time error demonstrating why `operator+` as a member function cannot take two explicit parameters.

3. Practice Questions (`03_PRACTICE_QUESTIONS/`)

- `conceptual_questions.md` (./03_PRACTICE_QUESTIONS/conceptual_questions.md)
- `predict_the_output.md` (./03_PRACTICE_QUESTIONS/predict_the_output.md)
- `debugging_questions.md` (./03_PRACTICE_QUESTIONS/debugging_questions.md)
- `programming_questions.md` (./03_PRACTICE_QUESTIONS/programming_questions.md)

4. Detailed Solutions (`04_DETAILED_SOLUTIONS/`)

- `conceptual_solutions.md` (./04_DETAILED_SOLUTIONS/conceptual_solutions.md)
- `output_solutions.md` (./04_DETAILED_SOLUTIONS/output_solutions.md)
- `programming_solutions.md` (./04_DETAILED_SOLUTIONS/programming_solutions.md)

5. Technical Diagrams (`05_DIAGRAMS/`)

- `diagram_01_complex_system_operator_plus.svg` (./05_DIAGRAMS/diagram_01_complex_system_operator_plus.svg)

- `diagram_02_member_vs_friend_overload.svg`
  (./05_DIAGRAMS/diagram_02_member_vs_friend_overload.svg)

01 — Operator Overloading: Fundamentals, Syntax, and Strict Rules

1. Definition & Intuition

Operator Overloading is a compile-time polymorphism feature in C++ that allows programmers to define custom meanings for existing C++ operators (`+` , `-` , `*` , `==` , `++` , `<<` , etc.) when applied to user-defined types (`class` or `struct`).

Why Operator Overloading is Needed

In Mathematics and Computing, we work with mathematical entities such as Complex Numbers, Matrices, Vectors, Polynomials, and Rational Fractions.

- Without operator overloading, adding two complex numbers requires verbose function calls: `c3 = c1.add(c2);`
- With operator overloading, we write natural mathematical notation: `c3 = c1 + c2;`

2. Syntax of an Overloaded Operator Function

```cpp
return_type operator@(parameter_list) {
    // implementation
}
```

Here `@` represents any overloadable C++ operator symbol (such as `+` , `-` , `*` , `++` , `==`).

3. Operators That CANNOT Be Overloaded (High-Frequency Exam Question)

The following C++ operators **cannot** be overloaded under any circumstances (`[over.oper]`):

1. **Scope Resolution Operator (`::`)** — Operates on compile-time names/scopes, not runtime values.
2. **Member Access / Dot Operator (`.`)** — Preserves unambiguous access to actual class members.
3. **Pointer-to-Member Operator (`.*`)** — Preserves built-in pointer-to-member resolution.
4. **Ternary Conditional Operator (`?:`)** — Has short-circuit evaluation semantics for its second and third operands that normal function argument evaluation cannot replicate.
5. **Object Size Operator (`sizeof`)** — Evaluated at compile time by the compiler.
6. **Type Identification Operator (`typeid`)** — Built-in RTTI mechanism.
7. **Alignment & Exception Operators (`alignof` , `noexcept`)** — Compile-time type/expression queries.

4. Operators That MUST Be Non-Static Member Functions

The C++ standard mandates that these four operators can **only** be overloaded as **non-static member functions** (they **cannot** be overloaded as `friend` or free non-member functions):

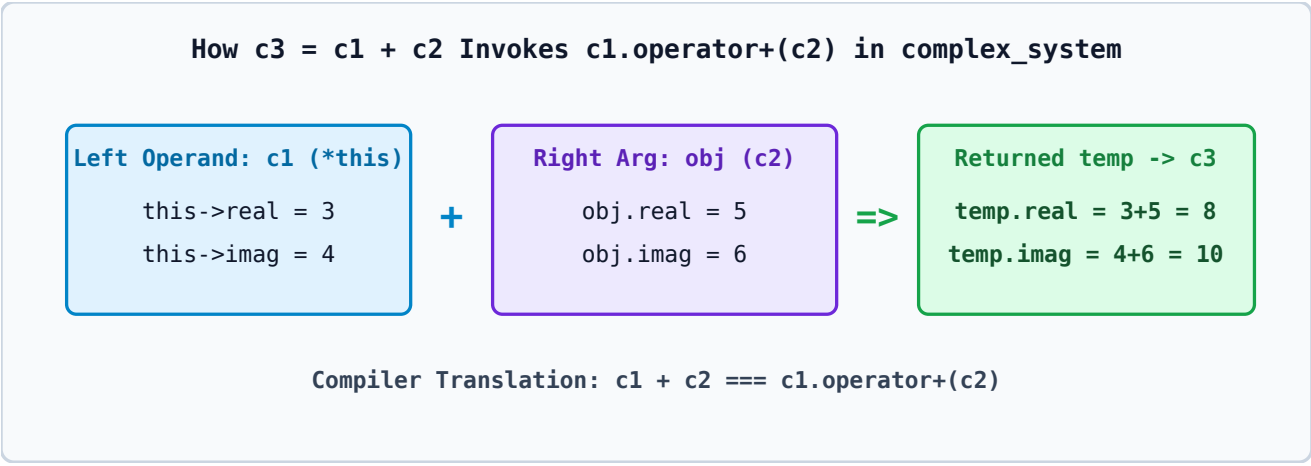
1. Assignment operator: `=`
2. Array subscript operator: `[]`
3. Function call operator: `()`
4. Member access arrow operator: `→`

---

5. Fundamental Rules & Restrictions of Operator Overloading

1. **No New Operator Symbols**: You cannot invent new tokens like `**` for exponentiation or `<>` for not-equal.
2. **Precedence and Associativity Never Change**: Overloading `*` and `+` for matrices will still evaluate `A + B * C` as `A + (B * C)` with the exact same built-in precedence and associativity.
3. **Arity (Number of Operands) Never Changes**: A unary operator cannot take two operands, and a binary operator cannot take three operands.
4. **No Default Arguments** (except for `operator()`): Overloaded operator functions cannot have default arguments (`operator+(int x = 0)` is a compilation error).
5. **At Least One User-Defined Operand**: You cannot redefine how `+` works on two built-in `int` values (`int operator+(int, int)` is illegal).

02 — Mandatory `complex_system` Example and Signed Integer Overflow Analysis



Complex System Operator Plus Diagram

1. Classroom `complex_system` Implementation

Below is the clean, classroom-style `complex_system` program demonstrating constructor initialization, `Show_Num()`, `operator+()`, `c1 + c2`, and the explicit member call `c1.operator+(c2)`:

CPP

```cpp
#include <iostream>
using namespace std;

class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) {
        real = r;
        imag = i;
    }

    complex_system operator+(const complex_system& obj) const {
        complex_system temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }

    void Show_Num() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    complex_system c1(3, 4);
    complex_system c2(5, 6);

    // 1. Implicit operator expression syntax:
    complex_system c3 = c1 + c2;

    // 2. Explicit member function call syntax:
    complex_system c4 = c1.operator+(c2);

    c1.Show_Num();
    c2.Show_Num();
    c3.Show_Num();
    c4.Show_Num();

    return 0;
}
```

2. How `c1 + c2` Maps to `c1.operator+(c2)`

When the compiler encounters the binary expression:

CPP

```cpp
c3 = c1 + c2;
```

1. **Left Operand (`c1`)**: Becomes the implicit invoking object (`*this` pointer inside `operator+`). Inside `operator+` , bare `real` and `imag` refer to `c1.real` (`this→real`) and `c1.imag` (`this→imag`).
2. **Right Operand (`c2`)**: Is passed as the single explicit argument `obj` (`const complex_system& obj`).
3. **Result (`temp`)**: A brand-new `complex_system` object is constructed with `temp.real = c1.real + c2.real` and `temp.imag = c1.imag + c2.imag` , and returned **by value** so neither `c1` nor `c2` is modified!
4. **Why `c1 + c2` and `c1.operator+(c2)` are 100% Equivalent**: The C++ compiler literally rewrites `c1 + c2` into the member function call `c1.operator+(c2)` .

---

3. Critical Edge Cases: Signed Integer Overflow & Negating `INT_MIN`

When working with integer-based `complex_system` objects, two important C++ standard rules regarding **Signed Integer Overflow** (`[expr.pre]`) must be understood:

A. Signed Integer Addition Overflow (`real + obj.real`) is Undefined Behavior

- In C++, **unsigned** integer arithmetic wraps around modulo 2^N, which is well-defined.
- However, **signed** integer overflow (for example, adding `INT_MAX + 1` , where `INT_MAX` is `2147483647` on 32-bit `int`) is **Undefined Behavior (UB)**!
- Even in C++20 and C++23, where the standard guarantees that signed integers use **two's complement representation** (`[basic.fundamental]`), overflowing a signed `int` during arithmetic (`+` , `-` , `*` , unary `-`) remains **Undefined Behavior**.

B. The `Show_Num()` Negative Imaginary Formatting Trap (`-imag` when `imag == INT_MIN`)

Suppose a student modifies `Show_Num()` to print negative imaginary numbers nicely as `3 - 4i` instead of `3 + -4i` by writing:

CPP

```cpp
// CAUTION: SUBTLE UNDEFINED BEHAVIOR ON INT_MIN!
void Show_Num_Unsafe() const {
    if (imag < 0) {
        cout << real << " - " << -imag << "i" << endl; // UB if imag == INT_MIN!
    } else {
        cout << real << " + " << imag << "i" << endl;
    }
}
```

Why is `-imag` Undefined Behavior when `imag == INT_MIN` ?

In 32-bit two's complement representation:

- The minimum representable signed 32-bit `int` is `INT_MIN = -2147483648` (-2^{31}).
- The maximum representable signed 32-bit `int` is `INT_MAX = +2147483647` ($+2^{31} - 1$).
- Notice the asymmetry! $+2147483648$ **cannot be represented** in a 32-bit signed `int` !
- Therefore, if `imag == INT_MIN` , evaluating `-imag` (or `std::abs(INT_MIN)`) overflows the maximum positive signed `int` and triggers **Undefined Behavior**!

How to Format Negative Imaginary Components Safely in C++23

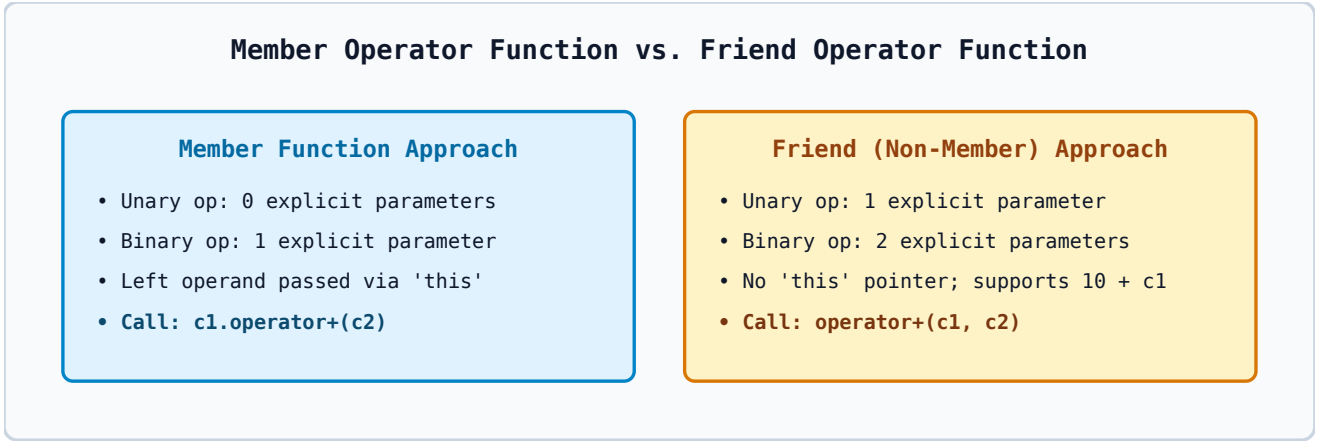
Either print `imag` directly (since `std::ostream::operator<<(int)` handles `INT_MIN` safely):

CPP

```cpp
void Show_Num() const {
    if (imag < 0) {
        cout << real << " - " << -(static_cast<long long>(imag)) << "i" << endl;
    } else {
        cout << real << " + " << imag << "i" << endl;
    }
}
```

Widening `imag` to a wider signed type (`long long`) or converting safely before negation avoids 32-bit `INT_MIN` negation UB.

03 — Unary vs. Binary Overloading and Member vs. Friend Approach



Member vs Friend Overload

1. Parameter Count Rules: Member Function vs. `friend` Function

Because a **non-static member function** always receives the left-hand (or sole) object implicitly via the hidden `this` pointer, it takes **one fewer explicit parameter** than a `friend` (non-member) function!

Operator Type	Expression	Member Function Signature	`friend` (Non-Member) Signature
Unary Generic (`-obj`)	`-c1`	`complex_system operator-() const;` (**0** params)	`friend complex_system operator-(const complex_system& a);` (**1** param)
Unary Prefix (`++obj`)	`++c1`	`counter& operator++();` (**0** params)	`friend counter& operator++(counter& a);` (**1** param)
Unary Postfix (`obj++`)	`c1++`	`counter operator++(int);` (**1 dummy** `int`)	`friend counter operator++(counter& a, int);` (**2** params)
Binary (`obj1 + obj2`)	`c1 + c2`	`complex_system operator+(const complex_system& b) const;` (**1** param)	`friend complex_system operator+(const complex_system& a, const complex_system& b);` (**2** params)

2. Distinguishing Prefix `++obj` from Postfix `obj++`

Both prefix and postfix increment use the exact same symbol `++`. How does the C++ compiler tell them apart?

- **Prefix (`++obj`)**: Declared with **no dummy parameter** (`counter& operator++()`). Increments the object first and returns `*this` by reference.
- **Postfix (`obj++`)**: Declared with an **unnamed dummy `int` parameter** (`counter operator++(int)`). Saves a copy of the old object, increments `*this`, and returns the saved old copy **by value**. When you write `obj++`, the compiler implicitly passes `0` as the dummy `int` argument (`obj.operator++(0)`).

---

3. When You MUST Use a `friend` (Non-Member) Function Over a Member Function

1. **When the Left-Hand Operand is a Built-In Type (`10 + c1`)**:
   - In `c1 + 10`, `c1` is on the left, so `c1.operator+(10)` works as a member function.
   - In `10 + c1`, the left operand is an `int` (`10`), and `int` is not a class (`10.operator+(c1)` is impossible!).
   - A `friend` function `friend complex_system operator+(int lhs, const complex_system& rhs)` takes both operands explicitly, allowing `10 + c1` to compile!
2. **Stream Insertion (`cout << c1`) and Extraction (`cin >> c1`)**:
   - In `cout << c1`, the left operand is `std::ostream` (`cout`), which belongs to the C++ Standard Library (`<iostream>`). Since we cannot add member functions inside `std::ostream`, `operator<<` and `operator>>` **must** be overloaded as non-member (typically `friend`) functions!

04 — Overloading Stream (`<<`, `>>`), Assignment (`=`), and Subscript (`[]`) Operators

1. Overloading `<<` and `>>` for Classroom Classes

```cpp
class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) : real(r), imag(i) {}

    friend ostream& operator<<(ostream& out, const complex_system& c) {
        out << c.real << " + " << c.imag << "i";
        return out; // Return ostream& to support chaining: cout << c1 << " " << c2;
    }

    friend istream& operator>>(istream& in, complex_system& c) {
        in >> c.real >> c.imag; // Note: 'c' is non-const reference so it can be modified!
        return in;
    }
};
```

Why Must `operator<<` Return `ostream&` by Reference?

1. Returning `ostream&` allows **chained output expressions** (`cout << c1 << endl;`), which groups left-to-right as `(cout << c1) << endl;` .
2. Furthermore, `std::ostream` is **non-copyable** (`std::ostream(const std::ostream&) = delete;`), so returning `ostream` by value is a compile-time error!

2. Overloading the Subscript Operator (`operator[]`)

The subscript operator `[]` must be a non-static member function and normally returns a **reference (`int&`)** so elements can be both read (`x = arr[0]`) and written (`arr[0] = 50`):

```cpp
int& operator[](int index) {
    return data[index];
}
```

3. Modern C++20 / C++23 Note: The Spaceship Operator (`⟺`)

In C++20 and C++23 (`#include <compare>`), defining `auto operator⟺(const MyClass&) const = default;` automatically synthesizes all six relational comparisons (`==` , `≠` , `<` , `⩽` , `>` , `⩾`) in a single line! In university exams, always write out explicit `bool operator==(const MyClass& other) const` unless `⟺` is specifically asked.

Visual Concept & Memory Diagrams (12 OPERATOR OVERLOADING)

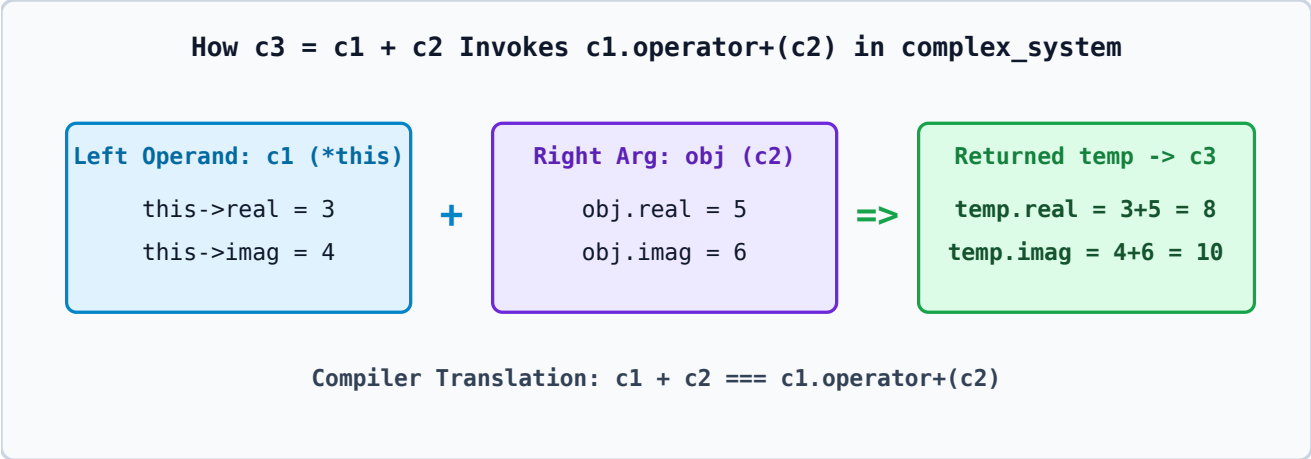


Diagram: diagram 01 complex system operator plus

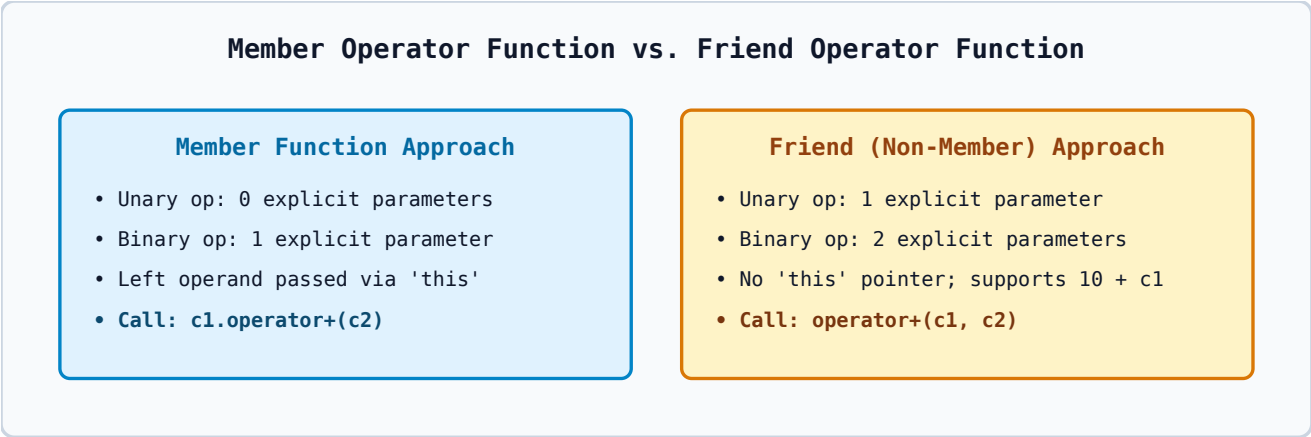


Diagram: diagram 02 member vs friend overload

Annotated C++ Programs (12 OPERATOR OVERLOADING)

Program: `01_classroom_complex_system.cpp`

C++ SOURCE FILE • 01_classroom_complex_system.cpp

```cpp
// Program Title: Mandatory Classroom Example — complex_system with Show_Num() and operator+
()
// Topic: Operator Overloading — Binary Member Operator Overloading
// Purpose: Demonstrate real/imaginary constructor initialization, Show_Num(), operator+(),
c1 + c2, and c1.operator+(c2).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_mandatory_complex_system_and_overflow_analysis.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) {
        real = r;
        imag = i;
    }

    complex_system operator+(const complex_system& obj) const {
        complex_system temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }

    void Show_Num() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    complex_system c1(3, 4);
    complex_system c2(5, 6);

    // 1. Adding two complex_system objects using natural '+' operator syntax:
    complex_system c3 = c1 + c2;

    // 2. Adding two complex_system objects using explicit member function call syntax:
    complex_system c4 = c1.operator+(c2);

    cout << "First complex number (c1): ";
    c1.Show_Num();
```

```
    cout << "Second complex number (c2): ";
    c2.Show_Num();

    cout << "Sum using c1 + c2 (c3): ";
    c3.Show_Num();

    cout << "Sum using c1.operator+(c2) (c4): ";
    c4.Show_Num();

    return 0;
}

/*
Expected Deterministic Output:
First complex number (c1): 3 + 4i
Second complex number (c2): 5 + 6i
Sum using c1 + c2 (c3): 8 + 10i
Sum using c1.operator+(c2) (c4): 8 + 10i
*/
```

Program: `02_complex_system_overflow_edge_case.cpp`

C++ SOURCE FILE • 02_complex_system_overflow_edge_case.cpp

```cpp
// Program Title: Safe Negative Imaginary Formatting vs. INT_MIN Signed Overflow Edge Case
// Topic: Operator Overloading — Signed Integer Overflow and Formatting Edge Cases
// Purpose: Explain why negating INT_MIN (-imag) is Undefined Behavior and how to print negative imaginary numbers safely.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_mandatory_complex_system_and_overflow_analysis.md
// Program Classification: Category A — Valid and well-defined program (demonstrating safe handling of INT_MIN)

#include <iostream>
#include <climits>
using namespace std;

class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) {
        real = r;
        imag = i;
    }

    // Safe Show_Num(): widens 'imag' to 'long long' before negating so that even INT_MIN is printed without UB!
    void Show_Num() const {
        if (imag < 0) {
            long long safe_abs_imag = -(static_cast<long long>(imag));
            cout << real << " - " << safe_abs_imag << "i" << endl;
        } else {
            cout << real << " + " << imag << "i" << endl;
        }
    }
};

int main() {
    complex_system c_normal(5, -7);
    complex_system c_edge(1, INT_MIN); // On 32-bit int, INT_MIN is -2147483648

    c_normal.Show_Num();
    c_edge.Show_Num();

    return 0;
}

/*
Expected Deterministic Output (assuming 32-bit int where INT_MIN == -2147483648):
5 - 7i
1 - 2147483648i
*/
```

Program: `03_unary_prefix_and_postfix_overload.cpp`

C++ SOURCE FILE • 03_unary_prefix_and_postfix_overload.cpp

```cpp
// Program Title: Unary Operator Overloading (Unary Minus, Prefix ++, and Postfix ++)
// Topic: Operator Overloading — Unary Operators
// Purpose: Demonstrate prefix ++obj, postfix obj++ (with dummy int), and unary -obj in a classroom class.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_unary_and_binary_member_vs_friend.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class counter {
private:
    int count;

public:
    counter(int c = 0) {
        count = c;
    }

    // Unary minus (-obj): returns a new counter with negated value
    counter operator-() const {
        return counter(-count);
    }

    // Prefix increment (++obj): increments first, returns updated object
    counter& operator++() {
        ++count;
        return *this;
    }

    // Postfix increment (obj++): dummy 'int' parameter distinguishes it from prefix ++
    counter operator++(int) {
        counter old = *this;
        count++;
        return old;
    }

    void display() const {
        cout << "Count: " << count << endl;
    }
};

int main() {
    counter c(10);

    counter c_pre = ++c;   // c becomes 11, c_pre is 11
    c_pre.display();

    counter c_post = c++;  // c_post gets old value 11, c becomes 12
    c_post.display();
    c.display();
```

```
    counter c_neg = -c;    // c_neg is -12
    c_neg.display();

    return 0;
}

/*
Expected Deterministic Output:
Count: 11
Count: 11
Count: 12
Count: -12
*/
```

Program: `04_friend_binary_and_stream_overload.cpp`

C++ SOURCE FILE • 04_friend_binary_and_stream_overload.cpp

```cpp
// Program Title: Overloading Binary + and Stream << Using Friend Functions
// Topic: Operator Overloading — Friend Operator Overloading
// Purpose: Show how friend functions enable symmetric addition (10 + c1) and stream output
(cout << c1).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_unary_and_binary_member_vs_friend.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) {
        real = r;
        imag = i;
    }

    friend complex_system operator+(const complex_system& a, const complex_system& b) {
        return complex_system(a.real + b.real, a.imag + b.imag);
    }

    friend ostream& operator<<(ostream& out, const complex_system& c) {
        out << c.real << " + " << c.imag << "i";
        return out;
    }
};

int main() {
    complex_system c1(2, 3);
    complex_system c2(4, 5);

    complex_system c3 = c1 + c2;
    complex_system c4 = 10 + c1; // Works because 10 converts to complex_system(10, 0) in
friend operator+!

    cout << "c3 = " << c3 << endl;
    cout << "c4 = " << c4 << endl;

    return 0;
}

/*
Expected Deterministic Output:
c3 = 6 + 8i
c4 = 12 + 3i
*/
```

Program: `05_invalid_operator_overload_compile_error.cpp`

C++ SOURCE FILE • 05_invalid_operator_overload_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Arity Error — Binary Member Operator Taking Two Explicit Parameters
// Topic: Operator Overloading — Member Function Parameter Count Rule
// Purpose: Demonstrate why a non-static member operator+ cannot declare two explicit
parameters.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/03_unary_and_binary_member_vs_friend.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) : real(r), imag(i) {}

    // COMPILE ERROR: A member binary operator+ already has 'this' as its left operand,
    // so it can take at most ONE explicit parameter!
    complex_system operator+(const complex_system& a, const complex_system& b) {
        return complex_system(a.real + b.real, a.imag + b.imag);
    }
};

int main() {
    complex_system c1(1, 2), c2(3, 4);
    complex_system c3 = c1 + c2;
    return 0;
}

/*
Why Compilation Fails:
Compiler diagnostic: "error: 'complex_system complex_system::operator+(const complex_system&,
const complex_system&)'
must have either zero or one argument".
Fix: Either remove the first parameter (making it a 1-parameter member function, as in
01_classroom_complex_system.cpp)
or add the 'friend' keyword (making it a 2-parameter non-member function, as in
04_friend_binary_and_stream_overload.cpp).
*/
```

Conceptual Questions — Module 12: Operator Overloading

Level 1 — Beginner

Q1. What is operator overloading in C++? List at least five C++ operators that **cannot** be overloaded and four operators that **must** be overloaded as non-static member functions.

Q2. In the classroom `complex_system` class, explain how the compiler interprets `c3 = c1 + c2;` when `operator+` is a member function vs. when `operator+` is a `friend` function.

Level 2 — Intermediate

Q3. Why does a binary operator overloaded as a member function require only **one** explicit parameter, whereas the same binary operator overloaded as a `friend` function requires **two** explicit parameters?

Q4. How does the C++ compiler differentiate between prefix increment (`++obj`) and postfix increment (`obj++`) during operator overloading? Why does prefix `++` typically return by reference (`T&`) while postfix `++` returns by value (`T`)?

Level 3 — Advanced

Q5. Why must the stream insertion operator (`<<`) and stream extraction operator (`>>`) be overloaded as non-member (often `friend`) functions rather than member functions of your class?

Level 4 — University Examination Level

Q6. Explain why signed integer overflow is **Undefined Behavior** in C++23 even though signed integers use two's complement representation. Specifically, if `complex_system::Show_Num()` formats negative imaginary components using `if (imag < 0) cout << real << " - " << -imag << "i";`, what happens when `imag == INT_MIN` and how do you fix it safely?

Debugging Questions — Module 12: Operator Overloading

Q9. Identify why the following `operator+` inside `complex_system` unintentionally mutates `c1` during `c3 = c1 + c2;`, and rewrite `operator+` correctly:

CPP

```cpp
class complex_system {
public:
    int real, imag;
    complex_system(int r = 0, int i = 0) : real(r), imag(i) {}
    complex_system operator+(const complex_system& obj) {
        real = real + obj.real;
        imag = imag + obj.imag;
        return *this;
    }
};
```

Q10. Identify why `complex_system c2 = 5 + c1;` fails to compile when `operator+` is a member function of `complex_system`, and show how to fix it.

Predict the Output Questions — Module 12: Operator Overloading

Level 1 — Beginner

Q7. Predict the output:

CPP

```cpp
#include <iostream>
using namespace std;

class complex_system {
    int real, imag;
public:
    complex_system(int r = 0, int i = 0) : real(r), imag(i) {}
    complex_system operator+(const complex_system& c) const {
        return complex_system(real + c.real, imag + c.imag);
    }
    void Show_Num() const {
        cout << real << "+" << imag << "i ";
    }
};

int main() {
    complex_system a(2, 5), b(3, 1);
    complex_system c = a + b + a;
    c.Show_Num();
    cout << endl;
    return 0;
}
```

Level 2 — Intermediate

Q8. Predict the output (Prefix vs. Postfix `++`):

CPP

```cpp
#include <iostream>
using namespace std;

class box {
    int val;
public:
    box(int v = 0) : val(v) {}
    box& operator++() {
        val += 5;
        return *this;
    }
    box operator++(int) {
        box temp = *this;
        val += 10;
        return temp;
    }
    void show() const { cout << val << " "; }
};

int main() {
    box b(10);
    box b1 = ++b;
    box b2 = b++;
    b1.show();
    b2.show();
    b.show();
    cout << endl;
    return 0;
}
```

Programming Questions — Module 12: Operator Overloading

Q11. Write a complete C++23 classroom program defining `class complex_system` with `operator+`, `operator-`, `operator*` (complex multiplication: $(a+bi)(c+di) = (ac - bd) + (ad + bc)i$), `operator==`, and `Show_Num()`.

Q12. Write a complete C++23 classroom program defining a `distance_measure` class (`feet` and `inches`) that overloads binary `+` (normalizing every `12` inches into `1` foot) and `<` (comparing two distances).

Detailed Solutions — Conceptual Questions (Module 12: Operator Overloading)

Solution to Q1

- **Cannot be overloaded**: `::` (scope resolution), `.` (member access), `.*` (pointer-to-member access), `?:` (ternary conditional), `sizeof`, `typeid`, `alignof`, `noexcept`.
- **Must be non-static member functions**: `=`, `[]`, `()`, `→`.

Solution to Q2 (`c3 = c1 + c2` Translation)

- **As a member function**: Rewritten by the compiler as `c3 = c1.operator+(c2);` where `&c1` is passed as `this`.
- **As a `friend` function**: Rewritten by the compiler as `c3 = operator+(c1, c2);` where both `c1` and `c2` are explicit arguments.

Solution to Q3 (1 Parameter vs. 2 Parameters)

- A non-static member function always receives the invoking left-hand object implicitly via `this`. Adding a second explicit parameter to a binary member operator would give it 3 total operands (1 implicit + 2 explicit), violating the binary arity of `+`. A `friend` function is not a member function, so it has no `this` pointer and must receive both operands explicitly.

Solution to Q4 (Prefix `++obj` vs. Postfix `obj++`)

- Prefix `++` has signature `T& operator++()` (0 parameters), mutates `*this`, and returns `*this` by reference (avoiding an unnecessary copy).
- Postfix `++` has signature `T operator++(int)` (1 dummy `int` parameter), copies `*this` into a local variable `old`, increments `*this`, and returns `old` by value (since `old` is a local variable that will be destroyed on return, returning it by reference would create a dangling reference!).

Solution to Q5 (`operator<<` and `operator>>`)

- In `cout << obj`, the left-hand operand is `std::ostream` (`cout`), not `obj`. If `operator<<` were a member function of `MyClass`, you would have to write `obj << cout;`! To keep `cout` on the left, `operator<<` must be a non-member function taking `(ostream&, const MyClass&)`.

Solution to Q6 (Signed Overflow UB & `INT_MIN` Negation in `Show_Num()`)

- In C++23 (`[expr.pre]`), signed integer arithmetic overflow is Undefined Behavior.
- In 32-bit two's complement arithmetic, `INT_MIN` is `-2147483648` while `INT_MAX` is `+2147483647`. Because `+2147483648` is outside the range of `int`, evaluating `-imag` when `imag == INT_MIN` overflows and causes Undefined Behavior.
- **Fix**: Cast `imag` to a wider signed integer (`long long`) before negating: `-(static_cast<long long>(imag))`.

Detailed Solutions — Predict the Output & Debugging (Module 12: Operator Overloading)

Solution to Q7

- `a + b + a` associates Left-to-Right as `(a + b) + a`.
- `a + b` produces `complex_system(2+3, 5+1) = (5, 6)`.
- `(5, 6) + a` produces `complex_system(5+2, 6+5) = (7, 11)`.
- **Guaranteed Output**:

```text
7+11i
```

Solution to Q8

- `b` starts with `val = 10`.
- `box b1 = ++b;` calls prefix `operator++()`: `b.val` becomes `15`, and `b1.val` is `15`.
- `box b2 = b++;` calls postfix `operator++(int)`: `b2` receives the old copy (`15`), while `b.val` becomes `15 + 10 = 25`.
- **Guaranteed Output**:

```text
15 15 25
```

Solution to Q9 (Debugging Mutating `operator+`)

- **Bug**: `real = real + obj.real;` modifies `this→real` (`c1.real`) directly! Binary `+` should never mutate its left operand `c1`.
- **Fix**: Construct and return a temporary `complex_system` object and mark the member function `const`:

```cpp
complex_system operator+(const complex_system& obj) const {
  return complex_system(real + obj.real, imag + obj.imag);
}
```

Solution to Q10 (Debugging `5 + c1`)

- **Bug**: Member `operator+` requires the left operand to be of type `complex_system` (`c1.operator+(5)`), so `5 + c1` fails because `5` is a built-in `int`.
- **Fix**: Declare `operator+` as a `friend` function taking two `const complex_system&` parameters (since `complex_system(int r = 0, int i = 0)` is a non-explicit converting constructor, `5` implicitly converts to `complex_system(5, 0)`).

Detailed Solutions — Programming Questions (Module 12:

Operator Overloading)

Solution to Q11 (`complex_system` with `+`, `-`, `*`, and `==`)

CPP

```cpp
#include <iostream>
using namespace std;

class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) {
        real = r;
        imag = i;
    }

    complex_system operator+(const complex_system& obj) const {
        return complex_system(real + obj.real, imag + obj.imag);
    }

    complex_system operator-(const complex_system& obj) const {
        return complex_system(real - obj.real, imag - obj.imag);
    }

    complex_system operator*(const complex_system& obj) const {
        int r = (real * obj.real) - (imag * obj.imag);
        int i = (real * obj.imag) + (imag * obj.real);
        return complex_system(r, i);
    }

    bool operator==(const complex_system& obj) const {
        return (real == obj.real) && (imag == obj.imag);
    }

    void Show_Num() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    complex_system c1(3, 2);
    complex_system c2(1, 4);

    complex_system sum = c1 + c2;
    complex_system diff = c1 - c2;
    complex_system prod = c1 * c2; // (3*1 - 2*4) + (3*4 + 2*1)i = -5 + 14i

    sum.Show_Num();
    diff.Show_Num();
    prod.Show_Num();
    cout << "Equal? " << (c1 == c2) << endl;
```

```
    return 0;
}
/*
Expected Output:
4 + 6i
2 + -2i
-5 + 14i
Equal? 0
*/
```

Solution to Q12 (`distance_measure` with Normalized `+` and `<`)

CPP

```cpp
#include <iostream>
using namespace std;

class distance_measure {
private:
    int feet;
    int inches;

public:
    distance_measure(int f = 0, int in = 0) {
        feet = f + (in / 12);
        inches = in % 12;
    }

    distance_measure operator+(const distance_measure& d) const {
        return distance_measure(feet + d.feet, inches + d.inches);
    }

    bool operator<(const distance_measure& d) const {
        int total1 = (feet * 12) + inches;
        int total2 = (d.feet * 12) + d.inches;
        return total1 < total2;
    }

    void display() const {
        cout << feet << " ft " << inches << " in" << endl;
    }
};

int main() {
    distance_measure d1(5, 9);
    distance_measure d2(3, 8);
    distance_measure d3 = d1 + d2; // 8 ft 17 in → normalized to 9 ft 5 in

    d3.display();
    cout << "Is d2 < d1? " << (d2 < d1) << endl;
    return 0;
}
/*
Expected Output:
9 ft 5 in
Is d2 < d1? 1
*/
```